

```
In [ ]: # Loading packages
import os
import pandas as pd
import numpy as np

# plotting packages
%matplotlib inline
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt
import matplotlib.cm as cm
import matplotlib.colors as clrs

#
from sklearn.cluster import KMeans
```

Load raw data

```
In [ ]: # Load raw data
raw = pd.read_csv('countryriskdata.csv')

#
print("Size of the dataset (row, col): ", raw.shape)
print("\nFirst 5 rows\n", raw.head(n=5))
```

Simple exploratory analysis

Print summary statistics

Note that all features have quite different variances, and Corruption and Legal are highly correlated.

```
In [ ]: #
print("\nSummary statistics\n", raw.describe().round(3))

#
print("\nCorrelation matrix\n")
corr = raw.corr()
corr.style.background_gradient(cmap='coolwarm').set_precision(3)
```

```
In [ ]: #
fig, axs = plt.subplots(2, 3, figsize=(12,6))
fig.suptitle('Scatter plot among each pair of the variables')
Labels = ['Corruption', 'Peace', 'Legal', 'GDP Growth']

#
m = 0
for i in range(len(Labels)-1):
    for j in range(1, len(Labels)-i):
        axs.flat[m].scatter(raw[Labels[i]], raw[Labels[i+j]]) #
        axs.flat[m].set_xlabel(Labels[i]) #
        axs.flat[m].set_ylabel(Labels[i+j]) #
        # Turn off tick labels.
        axs.flat[m].set_yticklabels([])
        axs.flat[m].set_xticklabels([])
        m = m+1
```

Plot histogram

Note that the distribution for GDP Growth is quite skewed.

```
In [ ]: #
plt.figure(1)
raw['Corruption'].plot(kind = 'hist', title = 'Corruption', alpha = 0.5)

plt.figure(2)
raw['Peace'].plot(kind = 'hist', title = 'Peace', alpha = 0.5)

plt.figure(3)
raw['Legal'].plot(kind = 'hist', title = 'Legal', alpha = 0.5)

plt.figure(4)
raw['GDP Growth'].plot(kind = 'hist', title = 'GDP Growth', alpha = 0.5)

plt.show()
```

K-means cluster

Pick features and normalization

Let's normalize all the features, effectively making them equally weighted.

Ref. [Feature normalization. \(https://stats.stackexchange.com/questions/21222/are-mean-normalization-and-feature-scaling-needed-for-k-means-clustering\)](https://stats.stackexchange.com/questions/21222/are-mean-normalization-and-feature-scaling-needed-for-k-means-clustering).

```
In [ ]: X = raw[['Corruption', 'Peace', 'Legal', 'GDP Growth']]
X = (X - X.mean()) / X.std()      #
print(X.head(5))
```

Perform elbow method

The marginal gain of adding one cluster dropped quite a bit from k=3 to k=4. We will choose k=3 (not a clear cut though).

Ref. [Determining the number of clusters in a dataset. \(https://en.wikipedia.org/wiki/Determining_the_number_of_clusters_in_a_data_set\)](https://en.wikipedia.org/wiki/Determining_the_number_of_clusters_in_a_data_set).

```
In [ ]: # https://stackoverflow.com/questions/41540751/sklearn-kmeans-equivalent-of-elbow-method
Ks = range(1,10)
inertia = [KMeans(i).fit(X).inertia_ for i in Ks]
fig = plt.figure()
plt.plot(Ks, inertia, '-bo')
plt.xlabel('Number of clusters')
plt.ylabel('Inertia (within-cluster sum of squares)')
plt.show()
```

K-means with $k = 3$

```
In [ ]: k = 3
kmeans = KMeans(n_clusters=k, random_state=0)
kmeans.fit(X)

#
print("inertia for k=3 is", np.round(kmeans.inertia_,3))

Cluster_Table = pd.DataFrame(data=kmeans.cluster_centers_, columns=['Corruption', 'Peace', 'Legal', 'GDP Growth'])
Cluster_Table['Cluster'] = Cluster_Table.index
print("\ncluster centers:\n")
print(Cluster_Table.to_string(index=False))

#
raw['Cluster Labels']=kmeans.labels_
pd.set_option("display.max_rows", None) #
raw
```

From the cluster centers, it appears that cluster 0 contains the countries with lowest risk, cluster 2 contains countries with medium risk and cluster 1 contains countries with highest risk

Visualize the result (2D plots)

```

In [ ]: # Scatter plots of different pairs of variables
fig, axs = plt.subplots(2, 3, figsize=(15,8))
fig.suptitle('Scatter plots for different pairs of variables')
Labels = ['Corruption', 'Peace', 'Legal', 'GDP Growth']
y = kmeans.labels_

#
norm = clrs.Normalize(vmin=0., vmax=y.max()+0.8)
cmap = cm.viridis

# Plot the Figure
m = 0
for i in range(len(Labels)-1):
    for j in range(1, len(Labels)-i):
        axs.flat[m].scatter(X[Labels[i]], X[Labels[i+j]], c=cmap(norm(y)), s=50)
        axs.flat[m].scatter(Cluster_Table[Labels[i]], Cluster_Table[Labels[i+j]], c='black', s=200, alpha=0.5)
    )
    axs.flat[m].set_xlabel(Labels[i])
    axs.flat[m].set_ylabel(Labels[i+j])

    axs.flat[m].set_yticklabels([]) #
    axs.flat[m].set_xticklabels([]) #
    m = m+1

```

Visualize the result (2D plots with names of countries)

plot country abbreviations instead of dots.

```
In [ ]: # Plot of the Scatter Figure among each pair
fig, axs = plt.subplots(2, 3, figsize=(15,8))
fig.suptitle('Scatter plot among each pair of the variables')
Labels=['Corruption','Peace','Legal','GDP Growth']
colors = ['blue','green', 'red']

# set up the color
norm = clrs.Normalize(vmin=0.,vmax=y.max()+0.8)
cmap = cm.viridis

#
m = 0
for i in range(len(Labels)-1):
    for j in range(1,len(Labels)-i):
        axs.flat[m].scatter(X[Labels[i]],X[Labels[i+j]],c=y, s=0, alpha=0)
        axs.flat[m].scatter(Cluster_Table[Labels[i]], Cluster_Table[Labels[i+j]], c='black', s=200, alpha=0.5
        )

        axs.flat[m].set_xlabel(Labels[i])
        axs.flat[m].set_ylabel(Labels[i+j])
        for h in range(X.shape[0]):
            axs.flat[m].text(X[Labels[i]].iloc[h], X[Labels[i+j]].iloc[h], raw['Abbrev'].iloc[h],
                            color=colors[y[h]], weight='semibold', horizontalalignment = 'center', verticalalignment =
                            'center')
        m = m+1
```

List the result

```
In [ ]: result = pd.DataFrame({'Country':raw['Country'], 'Abbrev':raw['Abbrev'], 'Label':y})
with pd.option_context('display.max_rows', None, 'display.max_columns', 3):
    print(result.sort_values(by=['Label', 'Country']))
```

Since Corruption and Legal are highly correlated, we drop the Corruption variable, i.e., we pick three features for this analysis, Peace, Legal and GDP Growth.

```
In [ ]: X = X[['Peace', 'Legal', 'GDP Growth']]
```

```
In [ ]: Ks = range(1, 10)
inertia = [KMeans(i).fit(X).inertia_ for i in Ks]

fig = plt.figure()
plt.plot(Ks, inertia, '-bo')
plt.xlabel('Number of clusters')
plt.ylabel('Inertia (within-cluster sum of squares)')
plt.show()
```

K-means with $k = 3$

```
In [ ]: k = 3
kmeans = KMeans(n_clusters=k, random_state=0)
kmeans.fit(X)

# print inertia & cluster center
print("inertia for k=3 is", np.round(kmeans.inertia_,3))

Cluster_Table = pd.DataFrame(data=kmeans.cluster_centers_, columns=['Peace', 'Legal', 'GDP Growth'])
Cluster_Table['Cluster'] = Cluster_Table.index
print("\ncluster centers:\n")
print(Cluster_Table.to_string(index=False))

#
raw['Cluster Lables']=kmeans.labels_
pd.set_option("display.max_rows", None) #
raw
```

It appears that Cluster 1 contains the countries with lowest risk, Cluster 0 contains the countries with medium risk, and Cluster 2 contains the countries with highest risk

Visualize the result (2D plots)

```

In [ ]: # Scatter plots of different pairs of variables
fig, axs = plt.subplots(1,3,figsize=(15,4))
fig.suptitle('Scatter plots for different pairs of variables')
Labels = ['Peace', 'Legal', 'GDP Growth']
y = kmeans.labels_

# set up the color
norm = clrs.Normalize(vmin=0.,vmax=y.max()+0.8)
cmap = cm.viridis

# Plot the Figure
m = 0
for i in range(len(Labels)-1):
    for j in range(1,len(Labels)-i):
        axs.flat[m].scatter(X[Labels[i]], X[Labels[i+j]], c=cmap(norm(y)), s=50)
        axs.flat[m].scatter(Cluster_Table[Labels[i]], Cluster_Table[Labels[i+j]], c='black', s=200, alpha=0.5)
    )
    axs.flat[m].set_xlabel(Labels[i])
    axs.flat[m].set_ylabel(Labels[i+j])
    # Turn off tick labels.
    axs.flat[m].set_yticklabels([])
    axs.flat[m].set_xticklabels([])
    m = m+1

```

Visualize the result (2D plots with names of countries)

plot country abbreviations instead of dots.


```

In [ ]: # Plot of the Scatter Figure among each pair
fig, axs = plt.subplots(1,3,figsize=(15,4))
fig.suptitle('Scatter plot among each pair of the variables')
Labels=['Peace','Legal','GDP Growth']
colors = ['blue','green', 'red']

# set up the color
norm = clrs.Normalize(vmin=0.,vmax=y.max()+0.8)
cmap = cm.viridis

# Plot the Figure
m = 0
for i in range(len(Labels)-1):
    for j in range(1,len(Labels)-i):
        axs.flat[m].scatter(X[Labels[i]],X[Labels[i+j]],c=y, s=0, alpha=0)
        axs.flat[m].scatter(Cluster_Table[Labels[i]], Cluster_Table[Labels[i+j]], c='black', s=200, alpha=0.5
    )
    axs.flat[m].set_xlabel(Labels[i])
    axs.flat[m].set_ylabel(Labels[i+j])
    for h in range(X.shape[0]):
        axs.flat[m].text(X[Labels[i]].iloc[h], X[Labels[i+j]].iloc[h], raw['Abbrev'].iloc[h],
            color=colors[y[h]], weight='semibold', horizontalalignment = 'center', verticalalignment =
'center')
    m = m+1

```

Visualize the result (3D plot)

```
In [ ]: # set up the color
norm = clr.Normalized(vmin=0.,vmax=y.max()+0.8)
cmap = cm.viridis

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

ax.scatter(X.iloc[:,0], X.iloc[:,1], X.iloc[:,2], c=cmap(norm(y)), marker='o')

centers = kmeans.cluster_centers_
ax.scatter(centers[:, 0], centers[:, 1], c='black', s=100, alpha=0.5)

ax.set_xlabel('Peace')
ax.set_ylabel('Legal')
ax.set_zlabel('GDP Growth')

plt.show()
```

List the result

```
In [ ]: result = pd.DataFrame({'Country':raw['Country'], 'Abbrev':raw['Abbrev'], 'Label':y})
with pd.option_context('display.max_rows', None, 'display.max_columns', 3):
    print(result.sort_values(by=['Label', 'Country']))
```